DJANGO 리뷰

스터디장 김준태

2026. 04. 24

Django 프로젝트 의 핵심 설정 파일들

1. Django MVT 아키텍처 **(**데이터 플로우**)**

```
┌──────────────────────────────────────────────────────────────┐
│                    Django MVT Architecture                   │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  [Client] ──────────(1. Request)──────────> [ urls.py ]      │
│  (Browser)                                       |           │
│      ^                                       (2. Routing)    │
│      |                                           v           │
│  (5. Response)                               [ views.py ]    │
│      |                                          (View)       │
│      |                                           |           │
│ [ Template ] <────(4. Context Data)──── (3. DB ORM Query)    │
│ (HTML/DTL)                                       v           │
│                                              [ models.py ]   │
│                                                (Database)    │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```
          Django 프로젝트 = 설정 파일들의 연결

gamelog/
├── config/                ← 프로젝트 설정 폴더
│   ├── settings.py        ← 모든 설정의 중심
│   ├── urls.py            ← URL 라우팅의 시작점
│   ├── wsgi.py            ← 웹 서버 연결 설정
│   └── asgi.py            ← 비동기 서버 설정
├── manage.py              ← Django 명령어 실행기
└── games/                 ← 우리가 만든 앱

──────────────────────────────────────────────────────────

settings.py에서 프로젝트명이 사용되는 곳:

ROOT_URLCONF = 'config.urls'        ← URL 설정 위치
WSGI_APPLICATION = 'config.wsgi.application'
                      ↑
          프로젝트 폴더명과 일치해야 함!
          오타나면 서버 실행 안 됨

──────────────────────────────────────────────────────────

INSTALLED_APPS = [
    'django.contrib.admin',     ← Django 기본 앱들
    'django.contrib.auth',
    ...
    'games',                    ← 우리가 만든 앱 등록
    'accounts',
]
```

| 코드 설명

config/settings.py

config/urls.py

|만약 코드가 틀렸다면?

[상황]
프로젝트를 'my_seccond_pjt'로 잘못 생성했고,
폴더명은 'my_second_pjt'로 고쳤지만 서버가 안 뜸

[에러 화면]
ModuleNotFoundError: No module named 'my_seccond_

```
# ═══════════════════════════════════════════════════
# 파일 1: settings.py
# ═══════════════════════════════════════════════════

# 수정 전 (에러)
ROOT_URLCONF = 'my_seccond_pjt.urls'
WSGI_APPLICATION = 'my_seccond_pjt.wsgi.application'

# 수정 후 (정상)
ROOT_URLCONF = 'my_second_pjt.urls'
WSGI_APPLICATION = 'my_second_pjt.wsgi.application'

# ═══════════════════════════════════════════════════
# 파일 2: wsgi.py
# ═══════════════════════════════════════════════════

# 수정 전
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'my_seccond_pjt.settings')

# 수정 후
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'my_second_pjt.settings')


# ═══════════════════════════════════════════════════
# 파일 3: asgi.py
# ═══════════════════════════════════════════════════

# 수정 전
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'my_seccond_pjt.settings')

# 수정 후
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'my_second_pjt.settings')


# ═══════════════════════════════════════════════════
# 파일 4: manage.py
# ═══════════════════════════════════════════════════

# 수정 전
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'my_seccond_pjt.settings')

# 수정 후
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'my_second_pjt.settings')
```

| Template과 URL 태그

```
            DTL = HTML 안에서 Python 로직 실행

┌──────────────────────────────────────────────────
│  {{ 변수 }}          → 값 출력
│  {% 태그 %}          → 로직 실행
│  {{ 변수|필터 }}     → 값 가공 후 출력
└──────────────────────────────────────────────────


──────────────────────────────────────────────────


URL 하드코딩 vs url 태그:

❌ <a href="/games/42/">상세</a>
   → URL 패턴 바뀌면 모든 템플릿 수정 필요

✅ <a href="{% url 'games:detail' pk=game상세</a>}">
   → URL 패턴 바뀌어도 자동 반영


──────────────────────────────────────────────────


URL 네임스페이스 (app_name):

games/urls.py:
app_name = 'games'          ← 네임스페이스 설정
path('<int:pk>/', views.detail, name='detail')

템플릿에서 사용:
{% url 'games:detail' pk=game.pk %}
        └──┬──┘  └──┬──┘
       app_name   url name
```

| 코드 설명

games/urls.py
templates/base.html (네비게이션 바)
games/templates/games/index.html

| 만약 코드가 틀렸다면?

[에러 상황]
{% url 'games:detail' %}에서 pk 인자를 빠뜨림

[에러 메시지]
NoReverseMatch at /
Reverse for 'detail' with no arguments not found.
1 pattern(s) tried: ['(?P<pk>[0-9]+)/']
'

```
<!-- 에러 코드: pk 인자 누락 -->

<a href="{% url 'games:detail' %}">{{ game.title }}</a>
<!--                            ↑ pk 인자가 없음! -->


<!-- 정상 코드: pk 인자 추가 -->

<!-- 방법 1: 위치 인자 -->
<a href="{% url 'games:detail' game.pk %}">{{ game.title }}</a>

<!-- 방법 2: 키워드 인자 (권장 - 더 명확함) -->
<a href="{% url 'games:detail' pk=game.pk %}">{{ game.title }}</a>
```

NoReverseMatch at /
→ URL을 역으로 찾으려는데 (이름 → 주소) 실패했다

Reverse for 'detail' with no arguments not found.
→ 'detail'이라는 URL을 찾는데 인자가 없어서 못 찾겠다

1 pattern(s) tried: ['(?P<pk>[0-9]+)/']
→ 이 패턴을 시도했는데, 여기 pk라는 인자가 필요하다

Model 정의와 필드 옵션

```
          Model → makemigrations → migrate
          (Python)    (설계도)      (실제 DB)

┌──────────────────────────────────────────────
│  class Game(models.Model):
│      title = models.CharField(max_length=200)
│      release_date = models.DateField(null=True,
│                                     blank=True)
└──────────────────────────────────────────────
                       ↓
┌──────────────────────────────────────────────
│  CREATE TABLE games_game (
│      id INTEGER PRIMARY KEY,
│      title VARCHAR(200) NOT NULL,
│      release_date DATE NULL
│  );
└──────────────────────────────────────────────


──────────────────────────────────────────────

필드 옵션: null vs blank

┌────────────────────┬────────────────────┐
│  blank=True        │  null=True         │
│  ────────────      │  ──────────        │
│  Form 유효성 검사  │  DB 스키마 레벨    │
│  "빈 값 제출 허용" │  "NULL 저장 허용"  │
│                    │                    │
│  Django가 처리     │  DB가 처리         │
└────────────────────┴────────────────────┘

문자열 필드: blank=True만 (빈 문자열 저장)
숫자/날짜:   null=True, blank=True 둘 다
```

코드 설명

games/models.py

만약 코드가 틀렸다면?

시나리오: 필드 옵션 수정

```python
# ═══════════════════════════════════════════════════
# 문제: URLField에 길이 제한 추가, 선택 필드 설정
# ═══════════════════════════════════════════════════

from django.core.validators import MinLengthValidator, MaxLengthValidator

class ExternalAPI(models.Model):
    # 수정 전: 길이 제한 없음
    api_url = models.URLField()

    # 수정 후: 최소 15자, 최대 60자 제한
    api_url = models.URLField(
        validators=[
            MinLengthValidator(15, message='URL은 최소 15자 이상이어야 합니다.'),
            MaxLengthValidator(60, message='URL은 최대 60자 이하여야 합니다.'),
        ]
    )

    # 수정 전: 필수 필드
    additional_info = models.TextField()

    # 수정 후: 선택 필드 (기본값 None)
    additional_info = models.TextField(
        null=True,      # DB에 NULL 허용
        blank=True,     # 폼에서 빈 값 허용
        default=None,   # 기본값 None
    )
```

Validator 종류

```python
from django.core.validators import (
    MinLengthValidator,     # 최소 길이
    MaxLengthValidator,     # 최대 길이
    MinValueValidator,      # 최소 값 (숫자)
    MaxValueValidator,      # 최대 값 (숫자)
    EmailValidator,         # 이메일 형식
    URLValidator,           # URL 형식
    RegexValidator,         # 정규표현식
)

# 사용 예시
price = models.DecimalField(
    max_digits=10,
    decimal_places=2,
    validators=[
        MinValueValidator(0, message='가격은 0 이상이어야 합니다.'),
        MaxValueValidator(99999999.99, message='가격이 너무 큽니다.'),
    ]
)
```

CRUD 구현 (View와 Template 연동)

CRUD = 모든 웹 서비스의 기본

```
CRUD = Create, Read, Update, Delete

┌──────────┬──────────┬──────────┬──────────
│ 동작     │ HTTP 메서드 │ URL 예시   │ View 함수
├──────────┼──────────┼──────────┼──────────
│ 목록 조회 │ GET      │ /        │ index
│ 상세 조회 │ GET      │ /1/      │ detail
│ 생성 폼   │ GET      │ /create/ │ create
│ 생성 처리 │ POST     │ /create/ │ create
│ 수정 폼   │ GET      │ /1/update/ │ update
│ 수정 처리 │ POST     │ /1/update/ │ update
│ 삭제 처리 │ POST     │ /1/delete/ │ delete
└──────────┴──────────┴──────────┴──────────

────────────────────────────────────────────

Create/Update View의 이중 요청 구조:

if request.method == 'POST':
    # 폼 데이터 처리 (저장)
else:
    # 빈 폼 또는 기존 데이터로 채워진 폼 표시

PRG 패턴 (Post-Redirect-Get):
POST 처리 후 redirect()로 다른 페이지로 이동
→ 새로고침 시 중복 제출 방지
```

코드 설명

Games/views.py
games/templates/games/detail.html (수정/삭제 링크)

만약 코드가 틀렸다면?

5차시 문제 - 수정/삭제 기능 구현

```python
# ═══════════════════════════════════════════════════
# 1. URL 추가 (games/urls.py)
# ═══════════════════════════════════════════════════

urlpatterns = [
    path('', views.index, name='index'),
    path('create/', views.create, name='create'),
    path('<int:pk>/', views.detail, name='detail'),

    # 추가할 URL
    path('<int:pk>/edit/', views.edit, name='edit'),      # 수정 폼
    path('<int:pk>/update/', views.update, name='update'), # 수정 처리
    path('<int:pk>/delete/', views.delete, name='delete'), # 삭제 처리
]


# ═══════════════════════════════════════════════════
# 2. View 함수 추가 (games/views.py)
# ═══════════════════════════════════════════════════

def edit(request, pk):
    """수정 폼을 보여주는 View (GET만 처리)"""
    game = get_object_or_404(Game, pk=pk)
    form = GameForm(instance=game)  # 기존 데이터로 폼 채우기
    return render(request, 'games/edit.html', {
        'form': form,
        'game': game,
    })


def update(request, pk):
    """수정 데이터를 처리하는 View (POST만 처리)"""
    game = get_object_or_404(Game, pk=pk)
    form = GameForm(request.POST, instance=game)
    if form.is_valid():
        form.save()
        return redirect('games:index')  # 수정 완료 후 목록으로
    # 유효성 검사 실패 시 다시 폼 표시
    return render(request, 'games/edit.html', {
        'form': form,
        'game': game,
    })


def delete(request, pk):
    """삭제 처리하는 View"""
    game = get_object_or_404(Game, pk=pk)
    game.delete()
    return redirect('games:index')  # 삭제 완료 후 목록으로
```

수정 폼 템플릿

```html
<!-- games/templates/games/edit.html -->
{% extends 'base.html' %}

{% block content %}
<h1>{{ game.title }} 수정</h1>

<!-- action: 데이터를 전송할 URL -->
<form action="{% url 'games:update' pk=game.pk %}" method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">수정</button>
</form>

<a href="{% url 'games:index' %}">취소</a>
{% endblock %}
```

ModelForm과 Widget

ModelForm = Model 기반 자동 폼 생성

```
ModelForm: Model의 필드를 기반으로 폼 자동 생성

┌──────────────────────────────────────────────────
│  class Game(models.Model):
│      title = models.CharField(max_length=200)
│      price = models.DecimalField(...)
│      is_free = models.BooleanField()
└──────────────────────────────────────────────────
                        ↓ 자동 변환
┌──────────────────────────────────────────────────
│  title    → <input type="text">
│  price    → <input type="number">
│  is_free  → <input type="checkbox">
└──────────────────────────────────────────────────

──────────────────────────────────────────────────

Widget: 필드가 HTML에서 어떻게 표시될지 결정

widgets = {
    'is_vegan': forms.RadioSelect(
        choices=[(True, 'Yes'), (False, 'No')]
    ),
    'description': forms.Textarea(attrs={'rows': 4}),
}

attrs: HTML 속성 추가
{'class': 'form-control', 'placeholder': '입력하세요'}
```

코드 설명

games/forms.py

만약 코드가 틀렸다면?

6차시 문제 - Widget 커스터마이징

```python
# ═══════════════════════════════════════════════════
# 문제 1: 가격 필드에 유효성 검사 추가
# ═══════════════════════════════════════════════════

class MenuForm(forms.ModelForm):
    class Meta:
        model = Menu
        fields = ['name', 'price', 'is_vegan']

        widgets = {
            'price': forms.NumberInput(attrs={
                'class': 'form-control',
                'step': '0.01',      # 소수점 2자리
                'min': '0',
                'max': '99999999',   # 8자리
            }),
        }

        error_messages = {
            'price': {
                'invalid': '올바른 가격을 입력해 주세요. 예: 12.34',
                'max_digits': '가격은 최대 8자리여야 합니다.',
                'decimal_places': '가격은 소수점 이하 두 자리여야 합니다.',
            },
        }


# ═══════════════════════════════════════════════════
# 문제 2: BooleanField를 RadioSelect로 변경
# ═══════════════════════════════════════════════════

class MenuForm(forms.ModelForm):
    # 필드 재정의 (choices 커스터마이징)
    is_vegan = forms.TypedChoiceField(
        coerce=lambda x: x == 'True',  # 문자열을 불리언으로 변환
        choices=[(True, 'Yes'), (False, 'No')],
        widget=forms.RadioSelect(attrs={
            'class': 'radio-inline',
        }),
        label='채식 메뉴',
    )

    class Meta:
        model = Menu
        fields = ['name', 'price', 'is_vegan']
```

주요 Widget 종류

```python
# 텍스트 입력
forms.TextInput()            # <input type="text">
forms.Textarea()             # <textarea>
forms.PasswordInput()        # <input type="password">

# 숫자
forms.NumberInput()          # <input type="number">

# 선택
forms.Select()               # <select> 드롭다운
forms.RadioSelect()          # 라디오 버튼
forms.CheckboxInput()        # 체크박스
forms.CheckboxSelectMultiple()  # 다중 선택 체크박스

# 날짜/시간
forms.DateInput()            # <input type="date">
forms.TimeInput()            # <input type="time">
forms.DateTimeInput()        # <input type="datetime-local">

# 파일
forms.FileInput()            # <input type="file">
forms.ClearableFileInput()   # 파일 + 삭제 체크박스
```

Static/Media 파일과 이미지 업로드

Static vs Media 파일

```
Static 파일 vs Media 파일

┌──────────────────────┬──────────────────────────
│      Static          │       Media
├──────────────────────┼──────────────────────────
│ 개발자가 미리 준비     | 사용자가 업로드           ||
| CSS, JS, 로고 이미지   | 게임 커버, 프로필 사진    ||
| 코드와 함께 배포       | 서버에만 저장            ||
| Git에 포함            | Git에 미포함 (.gitignore) |
| 변경 거의 없음         | 계속 추가/삭제           ||
└──────────────────────┴──────────────────────────


──────────────────────────────────────────────

파일 업로드 3요소 (하나라도 빠지면 실패):

1. settings.py
   MEDIA_URL = '/media/'
   MEDIA_ROOT = BASE_DIR / 'media'

2. Template
   <form method="POST" enctype="multipart/form-data">
                        ↑ 파일 전송에 필수!

3. View
   form = GameForm(request.POST, request.FILES)
                                  ↑ 파일 데이터 전달

⚠️ enctype 누락 시: 에러 없이 조용히 실패
```

코드 설명

config/settings.py (Media 설정)
config/urls.py (Media URL 추가)
games/models.py (ImageField)
games/views.py (파일 업로드 처리)
games/templates/games/create.html

만약 코드가 틀렸다면?

enctype 누락 에러 시연

```
<!-- 수정 전: enctype 없음 (에러) -->
<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">등록</button>
</form>

<!-- 결과: 폼은 제출되지만 이미지 업로드 안 됨 -->
<!-- DB에 cover_image 필드가 빈 값으로 저장됨 -->


<!-- 수정 후: enctype 추가 (정상) -->
<form method="POST" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">등록</button>
</form>
```

7차시 문제 - 날짜별 폴더 + 필터링

```
# games/models.py

class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()

    # 날짜별 폴더에 이미지 저장
    image = models.ImageField(
        upload_to='posts/%Y/%m/%d/',  # posts/2024/01/15/
        blank=True,
    )

    is_public = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

```
# games/views.py

def index(request):
    # GET 파라미터로 필터링
    is_public = request.GET.get('is_public')

    if is_public == 'False':
        # 비공개 게시글만
        posts = Post.objects.filter(is_public=False)
    elif is_public == 'True':
        # 공개 게시글만
        posts = Post.objects.filter(is_public=True)
    else:
        # 전체
        posts = Post.objects.all()

    return render(request, 'posts/index.html', {'posts': posts})
```

인증 시스템과 데코레이터

Django 인증 시스템

```
          Django가 제공하는 인증 기능

┌──────────────────────────────────────────────────
│  UserCreationForm     → 회원가입 폼 (자동 생성)
│  AuthenticationForm   → 로그인 폼 (자동 생성)
│  login(request, user) → 세션에 사용자 정보 저장
│  logout(request)      → 세션에서 사용자 정보 삭제
└──────────────────────────────────────────────────


──────────────────────────────────────────────────


데코레이터: 함수에 공통 기능 추가

@login_required
→ 비로그인 시 로그인 페이지로 리다이렉트
→ ?next= 파라미터로 원래 페이지 기억

@require_POST
→ POST 요청만 허용, GET 요청 시 405 에러
→ 삭제, 로그아웃 등 위험한 동작에 사용

@require_http_methods(['GET', 'POST'])
→ 지정된 메서드만 허용

사용법:
@login_required          # 1. 먼저 로그인 체크
@require_POST            # 2. 그 다음 메서드 체크
def delete(request, pk): # 둘 다 통과하면 실행
```

코드 설명

accounts/views.py
games/views.py (데코레이터 적용)

만약 코드가 틀렸다면?

8차시 문제 - 조건부 렌더링

```
<!-- templates/base.html -->

<nav>
    {% if user.is_authenticated %}
        <!-- 로그인 상태 -->
        <span>{{ user.username }}님</span>

        <!-- 로그아웃 버튼 -->
        <form action="{% url 'accounts:logout' %}" method="POST">
            {% csrf_token %}
            <button type="submit">로그아웃</button>
        </form>
    {% else %}
        <!-- 비로그인 상태 -->
        <p>로그인 해 주세요</p>
        <a href="{% url 'accounts:login' %}">로그인</a>
    {% endif %}
</nav>
```

시나리오 2: 본인만 버튼 표시S

```
<!-- 8차시 문제: 본인의 점수 증가 버튼만 표시 -->

{% for player in players %}
<div class="player-card">
    <span>{{ player.username }}: {{ player.score }}점</span>

    <!-- 로그인한 사용자와 대상 사용자가 같을 때만 버튼 표시 -->
    {% if user.username == player.username %}
        <form action="{% url 'accounts:score' pk=player.pk %}" method="POST">
            {% csrf_token %}
            <button type="submit">점수 +1</button>
        </form>
    {% endif %}
</div>
{% endfor %}
```

만약 코드가 틀렸다면?

시나리오 3: @require_POST 효과 시연

```
# 수정 전: @require_POST 없음 (보안 취약)
def logout(request):
    auth_logout(request)
    return redirect('games:index')

# → 주소창에 /accounts/logout/ 치면 로그아웃됨
# → <img src="/accounts/logout/"> 로 악용 가능


# 수정 후: @require_POST 추가 (안전)
@require_POST
def logout(request):
    auth_logout(request)
    return redirect('games:index')

# → 주소창에 /accounts/logout/ 치면 405 에러
# → POST 요청(폼 제출)만 허용
```